

Building a JPEG Decoder in C — A 6-Target Roadmap

This is a self-contained build plan based on the marker/Huffman/DCT pipeline you described. No code here — only what to build, why it has to exist, and the math behind it. You write the C.

Order matters. Each target produces data the next target consumes. Don't skip ahead even if it's tempting.

TARGET 1 — Marker Parsing Skeleton

What you're building: A loop that walks through the raw bytes of the file and identifies every marker, without yet understanding what most of them mean.

Why this has to exist first: A JPEG file is not "image data with a header." It's a *sequence of tagged segments*. You cannot find the quantization table, or the image width, or the compressed pixel data, unless you can reliably walk segment-by-segment. If your marker loop is wrong, every later target will silently read garbage. This is the part that has to be bulletproof before you touch anything else.

What a marker is (the only fact you need right now): Every marker is 2 bytes, and the first byte is always `0xFF`. The second byte tells you which marker it is. `0xFFD8` = Start of Image (SOI), `0xFFD9` = End of Image (EOI). Markers between SOI and EOI describe everything about the file.

Steps:

1. Open the file in binary mode. Read the first 2 bytes. Confirm they equal `0xFFD8`. If not, this isn't a valid JPEG — stop here.
2. Build a loop: read 2 bytes at a time. If the first byte isn't `0xFF`, you've lost sync with the file — treat it as corrupt (this will matter more once you reach compressed data, but for now, just detect it).
3. For markers you don't understand yet (anything that isn't SOI/EOI), you still need to skip them correctly. Most markers (not all — note this) are followed by a 2-byte **length** field. This length *includes the 2 length bytes themselves*. So the actual payload you need to skip is `length - 2` bytes.

4. Implement APPn handling (`0xFFE0–0xFFEF`) purely as "read length, skip that many bytes." You don't need to parse APPN content (JFIF/EXIF metadata) to decode the image — you just need to not get confused by it.
5. Stop your loop when you hit EOI, or when you hit a marker you'll handle in later targets (you'll come back and fill these in one by one).

Why the length field works the way it does: The length includes itself because the decoder needs to know "how many bytes do I advance from *this point*" — if it didn't include the 2 length bytes, every marker-length calculation in the spec would need a `+2` everywhere. Spec designers chose to bake that `+2` into the stored number once instead.

Self-test before moving on: Take a real JPEG, open it in a hex editor side by side with your program's output. Your program should print the correct sequence of marker names and correctly land exactly on the *next* `0xFF` byte every single time, for the whole file. If your byte position ever drifts, this is the bug to fix before Target 2 — everything downstream depends on this being exact.

TARGET 2 — Quantization Tables (DQT, `0xFFDB`)

What you're building: Parsing for the DQT marker, storing up to 4 quantization tables, each as a proper 8×8 grid.

Why this has to exist: JPEG compression works by throwing away high-frequency detail more aggressively than low-frequency detail (because the eye is less sensitive to it). It does this by *dividing* each DCT coefficient by a number before storing it, and that divisor grid is the quantization table. To reverse compression later (Target 6), you need these exact divisors to *multiply back*. Without this table, dequantization — and therefore the whole image — is mathematically meaningless.

Structure of the marker:

- 2-byte length (as always).
- Then, possibly *multiple* tables packed one after another in the same marker. Each table starts with **1 info byte**:
 - lower nibble = table ID (0–3)
 - upper nibble = precision: 0 means each of the 64 values is 1 byte, 1 means each is 2 bytes.
- Then 64 values for that table.
- Keep consuming tables from this marker until you've consumed exactly `length` bytes — there can legitimately be more than one table per DQT marker.

The math you need: zig-zag order. The 64 values are *not* stored row-by-row. They're stored in a diagonal zig-zag scan, because of how the table will be used later: the DCT coefficients are also produced in a low-frequency-to-high-frequency order, and zig-zag is the scan path that visits coefficients roughly in order of increasing frequency. This matters enormously once you get to Huffman/AC decoding (Target 5), because run-length-of-zeros encoding only works well if the zeros are clustered together — and high-frequency coefficients (which are usually zero) cluster together *along the zig-zag path*, not along a row-major path.

So: build a fixed lookup table, `zigzag position → row,col`. You can find/derive this as a standard 64-entry sequence — look up "JPEG zig-zag order" to get the exact index mapping, then write the assignment logic yourself. When you read the 64 values out of the file, value `#i` goes into position `zigzag[i]` of your 8×8 grid, not position `i` directly.

Steps:

1. Parse the info byte, get ID and precision.
2. Read 64 values (1 or 2 bytes each depending on precision).
3. Un-zig-zag them into an 8×8 array stored at `tables[ID]`.
4. Loop back if there's more data left in this marker (another table).

Self-test: Print table 0 (luminance) and table 1 (chrominance) as proper 8×8 grids. Luminance values should generally be smaller (less aggressive compression of brightness); chrominance values larger (color is compressed harder). If your numbers look randomly scattered or absurdly large, your zig-zag mapping is probably wrong.

TARGET 3 — Frame Header (SOF0, `0xFFC0`)

What you're building: Parsing of image geometry — width, height, components, and sampling — plus the concept of an **MCU** (Minimum Coded Unit), which is the actual grid you'll decode in.

Why this has to exist: Up to now you know *how the data is quantized*, but not *what shape the image is* or *how many color channels* it has. SOF0 is the only place that tells you. Crucially, it also tells you which quantization table belongs to which component — without this link, Target 6 won't know which table to apply to which channel's blocks.

Structure:

- Length (2 bytes).
- Precision (1 byte) — must be 8 for baseline JPEG. If it isn't, you're looking at a JPEG variant you haven't built support for yet.
- Height (2 bytes), Width (2 bytes).

- Number of components (1 byte): 1 = grayscale, 3 = YCbCr (the overwhelming majority of real-world JPEGs).
- For each component: component ID (1 byte), sampling factors (1 byte: upper nibble = horizontal, lower nibble = vertical), quantization table ID (1 byte).

Two real-world gotchas to handle (mentioned in your source material, worth taking seriously):

- Component IDs are *supposed* to be 1, 2, 3 (Y, Cb, Cr) but some real encoders emit 0, 1, 2. Detect ID 0 and shift everything by +1 internally so the rest of your code can always assume 1/2/3. If you don't handle this, otherwise-perfectly-valid files will fail.
- For now, only support sampling factor 1×1 for every component (no chroma subsampling). This is a deliberate simplification — handling 4:2:0/4:2:2 subsampling means each MCU contains more than one block per chroma channel, plus you'd need to upsample chroma later. Get the simple case fully working first; subsampling is a clean extension afterward, not a redesign.

The math: what is an MCU, and how many do you have? With 1×1 sampling, an MCU is just one 8×8 block for *each* component side-by-side (so for YCbCr: one Y block, one Cb block, one Cr block = one MCU). The image is divided into MCUs covering the full width and height, *rounded up*:

MCUs_across = ceil(width / 8)

MCUs_down = ceil(height / 8)

If width or height isn't a multiple of 8, the edge MCUs will decode extra padding pixels beyond the real image — that's expected and fine; you just crop them off when you save the final image in Target 6.

Steps:

1. Parse precision, height, width, component count.
2. Parse each component's ID, sampling, quant-table ID into a small per-component struct.
3. Apply the zero-based-ID fix if needed.
4. Reject the file (or flag it) if precision ≠ 8, sampling ≠ 1×1, or quant table ID is out of range — these are exactly the validations your source material emphasizes, because a parser that silently accepts garbage will fail confusingly three targets later instead of failing clearly here.
5. Compute and store MCUs_across / MCUs_down — you'll loop over this grid in Target 5/6.

Self-test: Print width, height, component count, and the quant-table ID assigned to each component. Cross-check width/height against the actual image (open it in any viewer).

TARGET 4 — Huffman Tables & Scan Setup (DHT, DRI, SOS)

This target has three markers because they're all "setup before the real decoding starts" — none of them touch pixel data yet.

4a. DHT (0xFFC4) — Define Huffman Table

Why this exists: The compressed data isn't stored as fixed-width numbers — it's stored as Huffman codes, which assign *shorter* bit-patterns to *more common* values to save space. To reverse that, you need the exact code-to-value mapping the encoder used, and that mapping is *not* a fixed standard — it's transmitted in the file itself, in this marker.

Structure:

- Length.
- 1 info byte: lower nibble = table ID (0–3), upper nibble = table class (0 = DC, 1 = AC).
- 16 bytes: `counts[1..16]`, where `counts[i]` = how many symbols use an *i*-bit-long code.
- Then, the symbols themselves, total count = sum of `counts[ ]`, listed in order of increasing code length.

The math: how do counts+symbols become actual binary codes? This is the part most people find non-obvious, so take it slowly. JPEG uses **canonical Huffman codes**, which means once you know how many codes exist of each length, the *codes themselves* are fully determined (you don't need to transmit them) — they're just assigned in increasing numeric order, in lock-step with increasing code length:

1. Start with `code = 0`.
2. For length = 1 up to 16:
 - Assign `code`, `code+1`, `code+2`, ... to however many symbols have *this* length (`counts[length]` of them), each symbol getting the next number up.
 - After finishing this length, do `code = code << 1` (shift left by one bit) before moving to the next length.

This shift-left step is exactly *why* shorter codes are never a prefix of longer codes (a property called "prefix-free" that makes the bitstream unambiguous): every time you move to a longer code length, you're conceptually appending a 0 bit to whatever range you'd reached, opening up fresh room rather than reusing old bit-patterns.

You don't need to store the codes as binary strings — store, for each code length, the *starting code value* and the *range of symbol-list indices* that belong to that length. A flat symbol array plus a 17-entry offset array (one entry per possible length 0–16, telling you where that length's symbols start in the flat array) is the right structure — this is what your source material describes, and it avoids a messy jagged 2D array.

Steps for 4a:

1. Parse table class and ID.
2. Read the 16 counts.
3. Read the symbols (total = sum of counts) into a flat array.
4. Build the offset array.
5. Store this as DC table **ID** or AC table **ID** depending on class.
6. Loop if there's more than one table in this marker (same pattern as DQT).

4b. DRI (0xFFDD) — Define Restart Interval

Why this exists: DC coefficients (next target) are encoded as a *difference* from the previous block's DC value, not as an absolute number — this saves bits because neighboring blocks usually have similar brightness. But this creates a fragile dependency chain: if you lose your place in the bitstream, every DC value after that point is wrong. DRI specifies how many MCUs to decode before resetting the DC difference back to zero — these reset points (marked by restart markers in the data) act as recovery checkpoints. Length is fixed at 4 (length field + 2-byte interval). Just read the 16-bit interval and store it; you'll use it in Target 5.

4c. SOS (0xFFDA) — Start of Scan

Why this exists: SOF told you what components exist; SOS tells you which *Huffman tables* each component uses — i.e., the missing link between "raw bits" and "which decoding rules to apply to them." After SOS, the file stops being markers-and-metadata and becomes the actual compressed bitstream.

Structure:

- Length.
- Number of scan components (for baseline, equal to the component count from SOF).
- For each: component ID, 1 byte where upper nibble = DC table ID to use, lower nibble = AC table ID to use.
- 3 more bytes (spectral selection start/end, successive approximation) — for baseline JPEG these are fixed (0, 63, 0); just validate they match, you don't need to use them for anything else.

The part that matters most: what comes immediately after SOS. Once you finish reading the SOS marker, you are now sitting at the start of raw compressed bytes — there is no more length field telling you how much data follows. You read it differently:

- Read byte by byte (you'll actually read *bit* by bit once decoding starts, in Target 5, but for now just locate where this segment ends).
- If you see `0xFF` followed by `0x00`, that `0x00` is not data — it's a deliberate insertion called **byte stuffing**: a literal `0xFF` *value* needs to be stored in the pixel data sometimes, but `0xFF` also means "marker coming," so the encoder always follows a genuine `0xFF` data byte with an extra `0x00` so the decoder can tell the difference. You strip that `0x00` out — the real data byte is just `0xFF`.
- If you see `0xFF` followed by `0xD0–0xD7`, that's a restart marker (these come at the intervals DRI specified) — it's not pixel data, it's a checkpoint signal.
- If you see `0xFF` followed by `0xD9`, that's EOI — the entropy-coded data, and the whole image, ends here.
- Multiple consecutive `0xFF` bytes can legally appear (this is a JPEG quirk — some encoders pad with extra `0xFF`s); don't treat repeats as an error.

Steps for 4c:

1. Parse component-to-table assignments.
2. Validate the fixed spectral-selection bytes.
3. Scan forward from this point, copying bytes into a buffer (removing stuffed `0x00`s) until you hit EOI, building yourself a clean buffer of "pure" compressed bits with no marker noise in it.

Self-test for all of Target 4: Print each Huffman table's symbol counts (DC tables are usually short, ~12 symbols max; AC tables can have up to 162). Print the size of the extracted entropy-coded buffer — your source material's example lands around 30 KB; yours should be a similarly plausible size, not near-zero or near the full file size.

TARGET 5 — Entropy (Huffman) Decoding

What you're building: The actual conversion of the compressed bit buffer (Target 4's output) into 64 DCT coefficients per block, per component, for every MCU in the image.

Why this exists: This is the bridge between "bits" and "numbers the math in Target 6 can use." Nothing before this point produced any actual image-related numbers — it was all bookkeeping. This is where decoding really begins.

Building block #1: a bit reader. Unlike everything before, this data is *not* byte-aligned — you need to read individual bits, most-significant-bit first, advancing through your cleaned buffer one bit at a time, crossing byte boundaries transparently. Build this as its own small reusable unit before anything else, because every other piece of this target depends on it.

Building block #2: matching bits to a Huffman symbol. Using the offset/symbol structure from Target 4: read one bit at a time, and after each bit, check whether the bits read *so far* match any valid code of that exact length (this is possible cheaply because canonical codes for a given length form a contiguous numeric range — recompute the starting code value per length the same way you did when building the table, and check if your accumulated value falls in range and corresponds to a real assigned symbol, not just an unused leftover code). Continue bit-by-bit until you find a match — prefix-free codes guarantee you'll never falsely match early.

Decoding one block's DC coefficient (math):

1. Decode one Huffman symbol using the component's DC table — this symbol is not the DC value, it's a **size** (how many more bits to read next).
2. If size is 0, the difference is 0.
3. Otherwise, read **size** more raw bits as an unsigned integer, call it **bits**. Apply the **receive-and-extend** rule used everywhere in JPEG for signed magnitudes:
 - If the *top* bit of **bits** is 1, the value is just **bits** (positive).
 - If the top bit is 0, the value is $\text{bits} - (2^{\text{size}} - 1)$ (negative). This works because JPEG doesn't use two's-complement here — it uses a symmetric "shifted" encoding where the bit pattern space is split: top half of the range = positive numbers, bottom half = negative, with no representation wasted on a redundant signed zero.
4. Add this difference to the *running* DC value for this component (reset to 0 at the very start of the scan, and again every time you cross a restart-interval boundary from Target 4b — this is exactly why that interval mattered). The result is this block's actual DC coefficient.

Decoding one block's 63 AC coefficients (math):

1. Starting at zig-zag position 1 (position 0 was DC), decode a Huffman symbol using the component's AC table.
2. Split this symbol into two nibbles: upper = **run** (how many zero coefficients precede the next nonzero one), lower = **size** (bit-length of the next nonzero coefficient — same receive-and-extend rule as DC applies here too).
3. Special cases: symbol **0x00** means **End of Block** — every remaining zig-zag position for this block is zero, stop immediately. Symbol **0xF0** means "skip 16 zeros, no coefficient follows" — advance your position by 16 and keep decoding.

4. Otherwise: skip **run** positions (leave them zero), decode **size** bits via receive-and-extend, place that value at the current zig-zag position, advance one more position, and repeat.
5. Stop when you've filled all 64 positions or hit End of Block.
6. Un-zig-zag this block into an 8×8 grid (same mapping as Target 2) — you now have raw, *quantized* DCT coefficients in proper 2D layout.

Steps to assemble the whole target:

1. Implement the bit reader.
2. Implement generic "decode one Huffman symbol given a table" using the bit reader.
3. Implement DC decode (steps above) and AC decode (steps above) as two functions built on top of symbol decoding.
4. Loop over every MCU position (from Target 3's MCU grid), and within each MCU, decode one block per component (3 blocks per MCU for YCbCr at 1×1 sampling), storing each block's 64 coefficients somewhere you can hand off to Target 6.
5. Handle restart markers: when you encounter one in the buffer, reset all components' DC predictors to 0 and discard any partial bits left in the current byte (restart markers are byte-aligned, so you must throw away unread bits in your current byte before continuing).

Self-test: Print the dequantized-but-still-DCT-domain DC value of the very first block. It should be a small-ish signed integer, not wildly out of range, and should not be all zeros across every block (that would mean your AC decode loop isn't actually consuming the run/size symbols correctly and is stuck re-reading garbage).

TARGET 6 — Inverse DCT, Dequantization, Color Conversion & Output

What you're building: The last stage — turning 64 quantized-DCT-coefficient blocks into actual RGB pixels, and saving a real image file.

Why this exists: Everything so far has been about *getting the right numbers*. This target is about understanding what those numbers physically mean: each block's 64 numbers describe a brightness pattern as a sum of cosine waves, not as direct pixel values. You have to run that description "backwards" to get pixels.

Step 1: Dequantization

This is simple and you've already built both halves of it. For each coefficient at position (u, v) in a block, multiply it by the value at (u, v) in the quantization table that the component's SOF entry pointed to (Target 3) and that you parsed in Target 2:

$$F(u,v) = \text{coefficient}(u,v) \times \text{quant_table}(u,v)$$

Do this for all 64 positions, for every block, before touching the IDCT.

Step 2: Inverse DCT (the core math)

The forward DCT (used during *encoding*, not something you're implementing) turns an 8×8 pixel block into 64 frequency coefficients. You need the inverse: given the 64 coefficients, reconstruct the 8×8 pixel block. The formula for one output pixel at position (x, y) within the block is:

$$f(x,y) = (1/4) \times \sum(u=0..7) \sum(v=0..7) C(u) \cdot C(v) \cdot F(u,v) \cdot \cos[(2x+1)u\pi/16] \cdot \cos[(2y+1)v\pi/16]$$

where $C(0) = 1/\sqrt{2}$ and $C(k) = 1$ for any $k > 0$.

In words: every output pixel is a weighted sum of all 64 coefficients, where each coefficient's contribution is shaped by a cosine wave whose frequency depends on (u, v) . $F(0, 0)$ (the DC coefficient) has no oscillation at all — it contributes the same amount to every pixel — which is exactly why it represents the *average brightness* of the block. Higher (u, v) coefficients oscillate faster and add finer detail.

Don't implement this formula literally as a 4-nested-loop (that's $O(N^4)$ per block and needlessly slow, plus harder to get right) — implement it *separably*. This is the practical trick that makes IDCT tractable:

- The 2D IDCT formula above factors into two passes of a 1D, 8-point IDCT: run a 1D inverse DCT across each of the 8 *rows* of the coefficient block, then run the same 1D inverse DCT down each of the 8 *columns* of the result (or columns-then-rows — order doesn't matter, the math is symmetric). This works because the 2D cosine term is literally a product of two independent 1D cosine terms — that's what "separable" means here.
- Write your 1D, 8-point inverse DCT as its own small function (8 inputs → 8 outputs), and call it 16 times per block (8 row-passes + 8 column-passes) instead of writing the full 2D version directly. This is both simpler to reason about and dramatically faster.

Step 3: Level shift and clamp

The forward DCT (during encoding) was applied to pixel values that had 128 *subtracted* from them first (to center the input range around 0, since DCT works best with signal centered at zero). So after your IDCT, **add 128 back** to every output value, then clamp the result into the valid 0–255 range (values can slightly overshoot due to rounding/precision in real-world coefficients — clamping is mandatory, not optional defensive coding).

You now have actual Y, Cb, Cr pixel values for every pixel in every MCU.

Step 4: YCbCr → RGB conversion

For every pixel:

$$R = Y + 1.402 \times (Cr - 128)$$

$$G = Y - 0.344136 \times (Cb - 128) - 0.714136 \times (Cr - 128)$$

$$B = Y + 1.772 \times (Cb - 128)$$

Clamp each of R, G, B to 0–255. (If your image has only 1 component — grayscale — skip this step entirely; R=G=B=Y.)

Why these specific constants: Y/Cb/Cr is a linear transformation of R/G/B designed so that Y alone carries (most of) the brightness information and Cb/Cr carry color, which is *why* JPEG can compress Cb/Cr more aggressively (Target 2's chrominance tables) without the eye noticing much loss — humans are far more sensitive to brightness detail than color detail. These three formulas are simply the matrix inverse of the R/G/B→YCbCr transform used during encoding.

Step 5: Assemble and save

1. Place each MCU's pixels into their correct position in a full-size width×height image buffer (remember: if width/height weren't multiples of 8, your last row/column of MCUs decoded extra padding pixels — crop those off now, don't include them in the saved image).
2. Save as a simple format you can implement quickly without another whole library — a raw PPM file (P6 format) is the standard choice here: a tiny plain-text header (P6\nwidth height\n255\n) followed by raw R,G,B bytes. Any image viewer can open a .ppm directly, and it requires no encoding library on your end.

Self-test (the real one — the whole project's payoff): Open your output .ppm next to the original .jpg in an image viewer. They should look like the same picture. If the image is recognizable but has color smearing or odd block edges, suspect the IDCT or the YCbCr

conversion. If it's pure static/noise, suspect Target 5 (Huffman decode) instead — by this point in the pipeline, garbage upstream always shows up as visual garbage, which is a genuinely satisfying debugging signal.

Quick map of the whole pipeline

SOI → [APPn skip] → DQT → SOF0 → DHT/DRI/SOS → [entropy-coded bytes] → EOI

|



Target 5: Huffman-decode into

64 quantized coefficients/block

|



Target 6: dequantize → IDCT → +128/clamp

→ YCbCr→RGB → assemble → save

Work through these in order, get a real self-test passing at each one, and don't move forward on a "probably fine" — the bugs in this kind of project are almost always upstream of where they appear to be.